# Big Data Storage & Retrieval

A Comparative Study of SQL and NoSQL Databases

Dmitrii Naumov

d.naumov@innopolis.university

March 2026

## Contents

**Abstract** **2**

**1 Introduction** **3**

**2 Dataset Description** **3**

**3 Data Cleaning** **4**

**4 Data Modeling** **4**

4.1 PostgreSQL — Relational Model . . . . . 4

4.2 MongoDB — Document Model . . . . . 6

4.3 Neo4j — Graph Model . . . . . 8

**5 Experimental Results** **9**

5.1 Test Environment . . . . . 9

5.2 Query 1 — Campaign Conversion and Social Targeting . . . . . 9

5.3 Query 2 — Personalised Product Recommendations . . . . . 11

5.4 Query 3 — Full-Text Search on Category Keywords . . . . . 13

5.5 Benchmarking . . . . . 14

**6 Discussion** **15**

6.1 Data Model Comparison . . . . . 15

6.2 Scalability Considerations . . . . . 16

6.3 Limitations . . . . . 16

**7 Conclusion** **16**

# Abstract

This report presents a comprehensive analysis of relational and non-relational database management systems for storing and querying big data. Data models for PostgreSQL, MongoDB, and Neo4j were designed and implemented. A real-world dataset consisting of user behaviour events, marketing campaigns, messages, client purchase history, and social network connections was loaded to data models. Three analytical queries are executed across all three databases to evaluate query expressiveness and performance. Benchmarking results show that all data model have their advantages and disadvantages. The study demonstrates that the choice of data model has a significant impact on both query complexity and execution time in big data scenarios.

# 1 Introduction

Modern e-commerce platforms generate data at high volume, variety, and velocity — the three core dimensions of big data [**?**]. Storing and querying such heterogeneous data efficiently requires careful selection of the underlying database paradigm.

This report investigates three representative database systems: PostgreSQL (relational), MongoDB (document-oriented), and Neo4j (graph). Each system have a different data model and offers distinct trade-offs between schema flexibility, query expressiveness, and runtime performance.

The dataset used in this study represents a real-world snapshot from an e-commerce platform. It contains five files: `events.csv`, `campaigns.csv`, `messages.csv`, `client_first_purchase_date` and `friends.csv`.

# 2 Dataset Description

The dataset consists of five CSV files described in Table 1.

Table 1: Dataset files overview.

| File | Rows | Description |
|---|---:|---|
| events.csv | 1,274,559 | User interactions with products |
| campaigns.csv | 1,908 | Marketing campaign metadata |
| messages.csv | 3,007,361 | Individual messages per recipient |
| client_first_purchase_date.csv | 175,013 | First purchase date per client |
| friends.csv | 1,974,239 | Mutual friendship pairs |

**events.csv** contains user behaviour records. Each row represents a single interaction between a user and a product (view, cart, purchase, or remove_from_cart), along with product metadata such as category, brand, and price.

**campaigns.csv** contains marketing campaigns across three types: bulk, trigger, and transactional. Campaigns are delivered via email, mobile push, web push, or SMS channels.

**messages.csv** contains individual messages sent to clients as part of campaigns. Each row records delivery metadata and engagement signals such as `is_opened`, `is_clicked`, and `is_purchased`.

**client_first_purchase_date.csv** records the date of each client's first purchase. A client identifier is formed by combining a fixed prefix with `user_id` and `user_device_id`.

**friends.csv** encodes mutual friendship relationships between users, forming the social graph used for targeted marketing analysis.

# 3 Data Cleaning

Prior to loading the data, a cleaning step was applied using `clean_data.py`. The following operations were performed per file.

**events.csv:** rows with missing `user_id`, `product_id`, or `event_time` were removed. Full duplicates and rows with unknown `event_type` values were dropped. Negative prices were removed. The `brand` and `category_code` fields were normalised to lowercase.

**campaigns.csv:** rows missing `id` or `campaign_type` were dropped. Duplicate `(id, campaign_type)` pairs and bulk campaigns with `finished_at` earlier than `started_at` were removed. Boolean columns were standardised from mixed string representations (`t/f`, `True/False`, `1/0`) to proper boolean values.

**messages.csv:** rows missing `client_id`, `campaign_id`, or `sent_at` were removed. Rows with logically impossible timestamps (e.g. `opened_first_time_at` before `sent_at`, `purchased_at` before `sent_at`) were dropped.

**client_first_purchase_date.csv:** rows missing `client_id` or `user_id` were removed. Duplicate `client_id` entries were deduplicated. Clients present in `messages.csv` but absent from this file were appended with `first_purchase_date = NULL` to preserve referential integrity across all three databases.

**friends.csv:** rows with missing values, self-loops (`friend1 = friend2`), and duplicate pairs were removed.

# 4 Data Modeling

## 4.1 PostgreSQL — Relational Model

The PostgreSQL schema follows third normal form (3NF). The dataset was decomposed into seven tables: `users`, `categories`, `products`, `events`, `campaigns`, `clients`, `messages`, and `friends`.

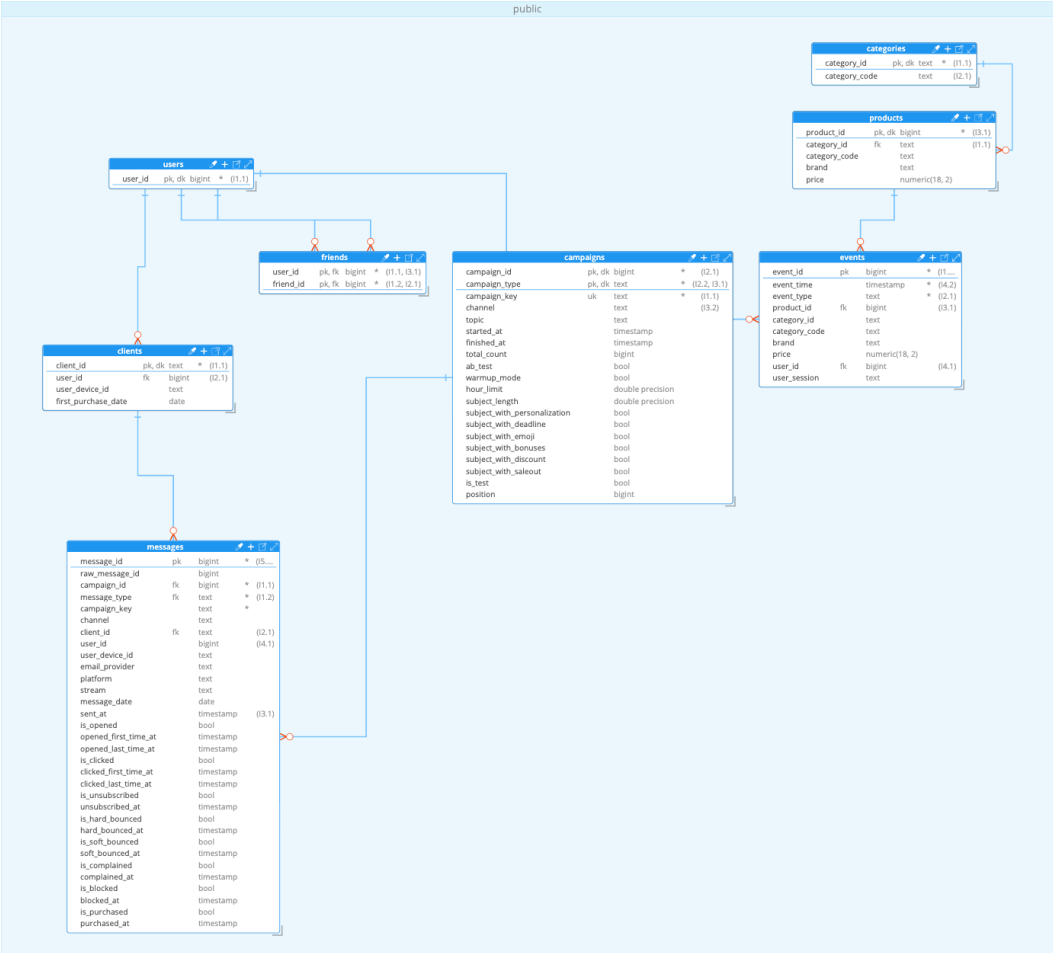


Figure 1: PostgreSQL entity-relationship diagram (Hackolade).

Key design decisions:

- `categories` is extracted as a separate table to avoid repeating the `category_code` string across millions of event rows.
- `clients` is separate from `users` because one `user_id` may map to multiple `client_id` values (one per device).
- The `messages` table does not use `campaign_id` as a foreign key alone, because `campaign_id` is only unique within a given `campaign_type`.
- The `friends` table enforces `CHECK (friend1 < friend2)` to eliminate symmetric duplicates, storing each pair once.

## 4.2 MongoDB — Document Model

The MongoDB model uses four collections: `events`, `campaigns`, `messages`, and `clients`. The design follows the MongoDB principle: *data that is accessed together should be stored together*.

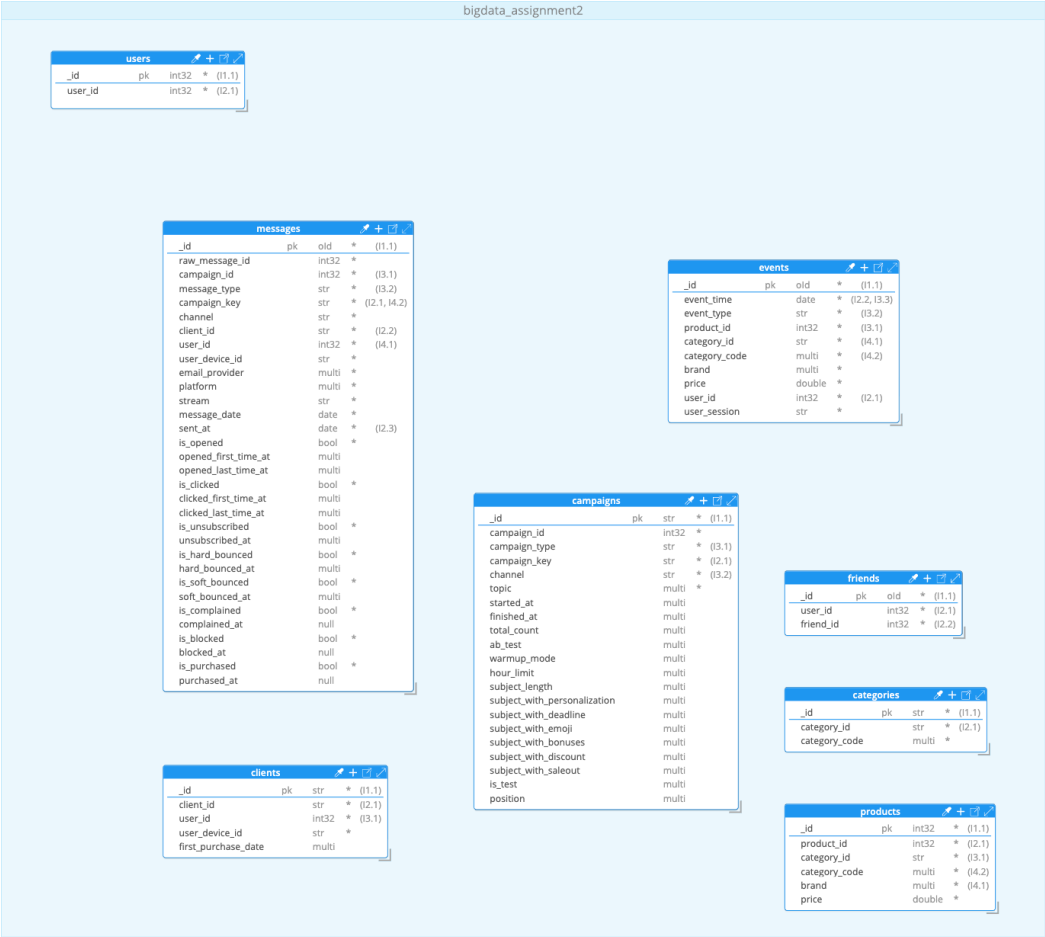


Figure 2: MongoDB collection schema (Hackolade).

Key design decisions:

- In `events`, product attributes are denormalised into an embedded `product` sub-document to eliminate `$lookup` on analytical queries.
- In `campaigns`, the seven `subject_with_*` flags are grouped into a nested `subject` sub-document.
- In `messages`, engagement metrics and delivery failure flags are separated into two sub-documents: `engagement` and `delivery`.
- Messages are *not* embedded inside campaigns because bulk campaigns can have over 48,000 recipients, exceeding MongoDB's 16 MB per-document limit.
- In `clients`, the friend list is embedded as an array of `user_id` values.

## 4.3 Neo4j — Graph Model

The Neo4j model represents entities as nodes and relationships as typed, property-carrying edges.

figures/model_neo4j.png

Figure 3: Neo4j graph schema.

The schema contains four node labels and four relationship types:

- **Nodes:** `User`, `Product`, `Category`, `Campaign`.
- **Relationships:** `FRIENDS_WITH` (User–User), `INTERACTED_WITH` (User–Product), `BELONGS_TO` (Product–Category), `RECEIVED_MESSAGE` (Client–Campaign).
- `FRIENDS_WITH` is stored as a directed edge; queries traverse both directions using the undirected pattern.
- The `price` at the time of the event is stored as a property on the `INTERACTED_WITH` edge, since prices change over time.

# 5 Experimental Results

## 5.1 Test Environment

Table 2: Hardware and software specifications.

| Component | Details |
|---|---|
| OS | macOS 15.3 Sequoia |
| CPU | Apple M4 Pro, 12 cores (8 performance + 4 efficiency) |
| RAM | 24 GB Unified Memory |
| PostgreSQL | 16.x (Docker container, `postgres:16`) |
| MongoDB | 7.x (Docker container, `mongo:7`) |
| Neo4j | 5.x Community (Docker container, `neo4j:5-community`) |
| Python | 3.12 (uv virtual environment) |

**Setup.** All three database engines were deployed as Docker containers on the host machine — no virtual machine or additional hypervisor layer was used. Docker Desktop for Mac (Apple Silicon) runs containers natively on the ARM architecture via the macOS Virtualization framework, so there is no instruction-set translation overhead. Each container was started with its default memory settings; no custom heap or buffer-pool configuration was applied. The benchmark script (`scripts/benchmark.py`) invokes the respective CLI tool for each database (`psql`, `mongosh`, `cypher-shell`) in a subprocess and measures wall-clock time using Python's `time.perf_counter()`. All three containers were running simultaneously on the same host during benchmarking, which may introduce I/O contention and inflate measured times relative to dedicated deployments.

## 5.2 Query 1 — Campaign Conversion and Social Targeting

**Approach.** The query is structured in three sub-parts executed sequentially. *Part 1* measures direct campaign effectiveness: for each `(campaign_id, type, channel)` combination, it aggregates the number of messages sent and computes open, click, purchase, and bounce rates. *Part 2* estimates incremental purchase lift: for every recipient's first campaign exposure it counts purchases in a 7-day window before and after, then computes the absolute lift as the difference between the two rates. *Part 3* performs social-network targeting: converted recipients (those who made a purchase after receiving a message) are identified, their friends who have not yet converted are ranked by the number of converted

friends they have and by the overlap between their browsed categories and the converter's purchased categories.

**Listing 1. PostgreSQL — Q1 Part 1 (campaign KPIs)**

```
WITH campaign_kpis AS (
    SELECT
        m.campaign_id, m.message_type,
        c.channel AS campaign_channel, c.topic, m.channel AS
           message_channel,
        COUNT(*) AS sent_count,
        SUM(CASE WHEN COALESCE(m.is_opened,    FALSE) THEN 1 ELSE 0
           END) AS opened_count,
        SUM(CASE WHEN COALESCE(m.is_clicked,   FALSE) THEN 1 ELSE 0
           END) AS clicked_count,
        SUM(CASE WHEN COALESCE(m.is_purchased, FALSE) THEN 1 ELSE 0
           END) AS purchased_count
    FROM messages AS m
    JOIN campaigns AS c
      ON c.campaign_id = m.campaign_id AND c.campaign_type =
         m.message_type
    GROUP BY m.campaign_id, m.message_type, c.channel, c.topic,
       m.channel
)
SELECT *,
    ROUND(opened_count::numeric    / NULLIF(sent_count, 0), 4) AS
       open_rate,
    ROUND(clicked_count::numeric   / NULLIF(sent_count, 0), 4) AS
       click_rate,
    ROUND(purchased_count::numeric / NULLIF(sent_count, 0), 4) AS
       purchase_rate
FROM campaign_kpis
ORDER BY purchase_rate DESC, purchased_count DESC
LIMIT 50;
```

Parts 2 and 3 of the PostgreSQL query use additional CTEs: `recipient_first_touch` computes the earliest message timestamp per (campaign, user) pair, `recipient_lift` applies correlated `EXISTS` subqueries against the `events` table for the 7-day windows, and `friend_candidates` / `scored_candidates` perform a self-join on the `friends` table to find and rank candidate targets.

**Listing 2. MongoDB — Q1 Part 1 (campaign KPIs)**

```
db.messages.aggregate([
  { $lookup: {
      from: "campaigns",
      let: { cid: "$campaign_id", mtype: "$message_type" },
      pipeline: [{ $match: { $expr: { $and: [
          { $eq: ["$campaign_id",   "$$cid"]   },
          { $eq: ["$campaign_type", "$$mtype"] }
      ]}}}],
      as: "campaign"
  }},
  { $unwind: "$campaign" },
  { $group: {
      _id: { campaign_id: "$campaign_id", message_type: "$message_type",
             campaign_channel: "$campaign.channel", message_channel:
                "$channel" },
```

```
      sent:      { $sum: 1 },
      purchased: { $sum: { $cond: [{ $eq: ["$engagement.is_purchased",
         true]}, 1, 0] }}
  }},
  { $addFields: {
      purchase_rate: { $round: [{ $divide: ["$purchased", { $max:
         ["$sent", 1] }] }, 4] }
  }},
  { $sort: { purchase_rate: -1 } },
  { $limit: 50 }
])
```

Parts 2 and 3 of the MongoDB script use JavaScript processing: purchase events are loaded into a `Map` keyed by `user_id` for O(1) window look-ups (Part 2), and the social targeting (Part 3) iterates over the embedded `clients.friends` array, checking membership in the pre-built `convertedSet` and counting category overlap via `userCategoryMap`.

**Listing 3. Neo4j Cypher — Q1 Part 1 (campaign KPIs)**

```
MATCH (cl:Client)-[m:RECEIVED_MESSAGE]->(c:Campaign)
WITH m.campaign_id AS campaign_id, m.message_type AS message_type,
     c.channel AS campaign_channel, m.channel AS message_channel,
     count(m) AS sent_count,
     sum(CASE WHEN coalesce(m.is_purchased, false) THEN 1 ELSE 0 END)
        AS purchased_count
RETURN campaign_id, message_type, campaign_channel, message_channel,
       sent_count, purchased_count,
       CASE WHEN sent_count > 0
            THEN round(10000.0 * toFloat(purchased_count) / sent_count)
               / 10000.0
            ELSE null END AS purchase_rate
ORDER BY purchase_rate DESC, purchased_count DESC
LIMIT 50;
```

Part 3 of the Cypher query uses a correlated `CALL` sub-query per candidate node to collect the converter's purchased categories and the candidate's browsed categories, then computes the set intersection size inline using a list comprehension: `size([cat IN converter_categories WHERE cat IN candidate_categories])`.

## 5.3 Query 2 — Personalised Product Recommendations

**Approach.** The query implements a content-based filtering algorithm combined with global product popularity boosting. It first selects the most active user (highest total event count) as the target. It then computes *category affinity scores* for that user by summing weighted event scores per category: purchase = +5, cart = +3, view = +1, remove_from_cart = −1. Global *product popularity scores* are computed identically across all users. Candidate products from the user's preferred categories that the user has not yet purchased are scored as score = affinity × 100 + popularity and the top-10 are returned.

**Listing 4. PostgreSQL — Q2 (recommendation CTEs)**

```
WITH target_user AS (
    SELECT user_id FROM events WHERE user_id IS NOT NULL
    GROUP BY user_id ORDER BY COUNT(*) DESC LIMIT 1
),
```

```
user_category_affinity AS (
    SELECT tu.user_id AS target_user_id,
           COALESCE(p.category_id, e.category_id) AS category_id,
           SUM(CASE e.event_type
               WHEN 'purchase' THEN 5 WHEN 'cart' THEN 3
               WHEN 'view'     THEN 1 WHEN 'remove_from_cart' THEN -1
               ELSE 0 END) AS category_affinity_score
    FROM target_user AS tu
    JOIN events AS e ON e.user_id = tu.user_id
    LEFT JOIN products AS p ON p.product_id = e.product_id
    GROUP BY tu.user_id, COALESCE(p.category_id, e.category_id), ...
    HAVING category_affinity_score > 0
),
product_popularity AS (
    SELECT e.product_id,
           SUM(CASE e.event_type WHEN 'purchase' THEN 5 ... END)
               AS product_popularity_score
    FROM events AS e LEFT JOIN products AS p ON ...
    GROUP BY e.product_id, ...
)
SELECT pp.product_id, pp.brand, pp.price, pp.category_code,
       uca.category_affinity_score * 100 + pp.product_popularity_score
           AS recommendation_score
FROM user_category_affinity AS uca
JOIN product_popularity AS pp ON pp.category_id = uca.category_id
LEFT JOIN purchased_products AS bought ON bought.product_id =
   pp.product_id
WHERE bought.product_id IS NULL
ORDER BY recommendation_score DESC
LIMIT 10;
```

**Listing 5. MongoDB — Q2 (aggregation outline)**

```
// 1. Find most active user
db.events.aggregate([
  { $group: { _id: "$user_id", event_count: { $sum: 1 } } },
  { $sort: { event_count: -1, _id: 1 } },
  { $limit: 1 }
])
// 2. Compute category affinities for target user
db.events.aggregate([
  { $match: { user_id: targetUserId } },
  { $lookup: { from: "products", localField: "product_id", ... } },
  { $addFields: { weight: { $switch: { branches: [
      { case: { $eq: ["$event_type", "purchase"] }, then: 5 }, ...
  ]}}}}
  { $group: { _id: "$resolved_category_id",
              category_affinity_score: { $sum: "$weight" } } }
])
// 3. Compute global product popularity (same weight scheme over all
   events)
// 4. Join in JavaScript: for each affinity category, find candidate
   products,
//    exclude already-purchased, rank by affinity*100 + popularity
```

**Listing 6. Neo4j Cypher — Q2**

```
CALL { MATCH (u:User)-[e:INTERACTED_WITH]->(:Product)
```

```
       RETURN u.user_id AS target_user_id, count(e) AS event_count
       ORDER BY event_count DESC LIMIT 1 }
MATCH (target:User {user_id: target_user_id})
CALL { WITH target
       MATCH (target)-[e:INTERACTED_WITH]->(p:Product)
       WITH coalesce(p.category_id, e.category_id) AS category_id,
            sum(CASE e.event_type WHEN 'purchase' THEN 5 WHEN 'cart'
               THEN 3
                WHEN 'view' THEN 1 WHEN 'remove_from_cart' THEN -1 END)
                   AS affinity
       WHERE affinity > 0
       RETURN collect({ category_id: category_id, affinity: affinity })
          AS affinities }
UNWIND affinities AS affinity
CALL { WITH affinity
       MATCH (:User)-[e:INTERACTED_WITH]->(p:Product)
       WHERE coalesce(p.category_id, e.category_id) =
          affinity.category_id
       WITH p, sum(CASE e.event_type WHEN 'purchase' THEN 5 ... END) AS
          popularity
       RETURN collect({ product_id: p.product_id, popularity:
          popularity, ... }) AS candidates }
UNWIND candidate_products AS rec
WHERE NOT rec.product_id IN purchased_product_ids
RETURN target_user_id, rec.product_id AS product_id, rec.brand AS brand,
       affinity.affinity * 100 + rec.popularity AS recommendation_score
ORDER BY recommendation_score DESC
LIMIT 10;
```

## 5.4 Query 3 — Full-Text Search on Category Keywords

**Approach.** Query 3 extends Query 2 by using the top-5 recommended category codes as a free-text search query. Each `category_code` is normalised by replacing dots, underscores, and hyphens with spaces (e.g. `electronics.audio.headphones` becomes `electronics audio headphones`). The concatenated keyword string is submitted to a full-text index on the `products` table/collection, returning products whose `category_code` or `brand` matches any of the keywords, ranked by relevance score. This simulates a search-engine-style product discovery flow driven by the user's inferred interests.

**Listing 7. PostgreSQL — Q3 (full-text search)**

```
-- Build keyword query from top-5 recommended categories
WITH search_terms AS (
    SELECT string_agg(DISTINCT
                trim(regexp_replace(category_code, '[._-]+', ' ', 'g')),
                   ' ')
           AS query_text
    FROM top_recommendations            -- derived from Q2 logic above
    WHERE category_code IS NOT NULL
)
SELECT st.query_text, p.product_id, p.brand, p.price, p.category_code,
       ts_rank_cd(
           to_tsvector('simple',
               coalesce(p.category_code,'') || ' ' ||
               regexp_replace(coalesce(p.category_code,''),'[._-]+','
                  ','g') || ' ' ||
```

```
                coalesce(p.brand,'')),
            websearch_to_tsquery('simple', st.query_text)
        ) AS search_rank
FROM search_terms AS st
JOIN products AS p
  ON to_tsvector('simple', ...) @@ websearch_to_tsquery('simple',
     st.query_text)
ORDER BY search_rank DESC
LIMIT 20;
```

**Listing 8. MongoDB — Q3 (text index search)**

```
// Normalise top-5 category codes into a keyword string
const queryText = topCategories
    .map(c => String(c).replace(/[._-]+/g, ' ').trim())
    .join(' ');
// Submit to the text index on { category_code: "text", brand: "text" }
db.products.find(
  { $text: { $search: queryText } },
  { score: { $meta: "textScore" }, product_id: 1, brand: 1,
     category_code: 1 }
).sort({ score: { $meta: "textScore" }, product_id: 1 }).limit(20)
```

**Listing 9. Neo4j Cypher — Q3 (full-text index)**

```
-- Build keyword string from top-5 recommendations (reuses Q2 logic)
WITH trim(reduce(q = '', row IN top_recommendations |
    q + ' ' + replace(replace(replace(
        coalesce(row.category_code,''), '.', ' '), '_', ' '), '-', ' ')
)) AS query_text
CALL { WITH query_text WHERE query_text <> ''
       CALL db.index.fulltext.queryNodes('productFulltext', query_text)
           YIELD node, score
       RETURN node, score }
RETURN target_user_id, query_text,
       node.product_id AS product_id, node.brand AS brand,
       node.category_code AS category_code, score AS search_rank
ORDER BY search_rank DESC, product_id ASC
LIMIT 20;
```

## 5.5 Benchmarking

Each query was executed 5 times per database using `scripts/benchmark.py`. The script measures wall-clock time via Python's `time.perf_counter()` and includes the CLI process start-up overhead (negligible for `psql` but measurable for `mongosh`). Mean execution time and standard deviation are reported in Table 3 and visualised in Fig. 4.

Table 3: Query execution times (seconds), mean ± std over 5 runs.

| Query | Database | Mean (s) | Std (s) |
|---|---|---|---|
| Q1 | PostgreSQL | 12.56 | 0.45 |
| Q1 | MongoDB | | [TBD] |
| Q1 | Neo4j | 37.05 | 3.57 |
| Q2 | PostgreSQL | 1.03 | 0.007 |
| Q2 | MongoDB | 7.72 | 0.37 |
| Q2 | Neo4j | 8.2 | 0.4 |
| Q3 | PostgreSQL | 0.98 | 0.023 |
| Q3 | MongoDB | 7.44 | 0.043 |
| Q3 | Neo4j | 7.8 | 0.08 |

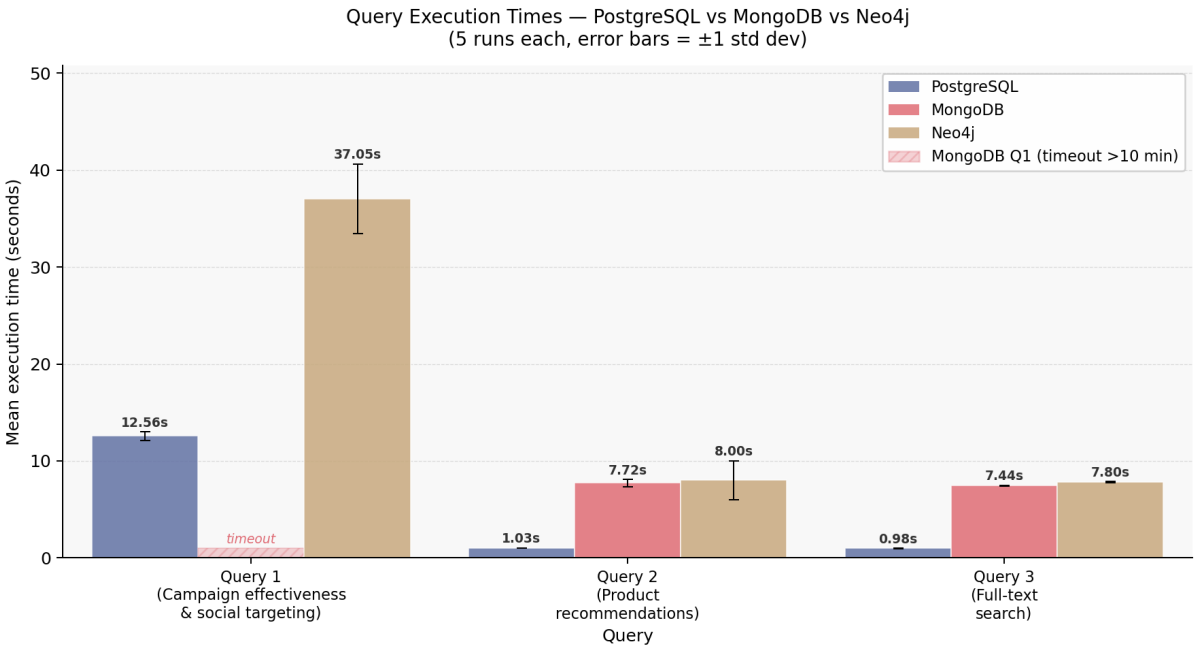


Figure 4: Query execution times across databases (5 runs each).

# 6 Discussion

## 6.1 Data Model Comparison

PostgreSQL enforces strict schema and referential integrity via foreign keys, making it well suited for structured transactional data. The normalised schema eliminates redundancy but requires multi-table joins for analytical queries, which become costly at scale.

MongoDB's document model allows denormalisation of related data — embedding product details inside event documents avoids joins entirely. However, the absence of enforced referential integrity means data consistency must be maintained at the application layer.

Neo4j excels at relationship-centric queries. The social targeting sub-query — finding friends of buyers not yet reached by any campaign — requires only a two-hop graph

traversal in Cypher. The equivalent SQL query requires two self-joins on the `friends` table combined with a `NOT IN` subquery, and the MongoDB equivalent requires a multi-stage `$lookup` pipeline or client-side iteration.

### 6.2 Scalability Considerations

At petabyte scale, all three systems would require horizontal scaling strategies. PostgreSQL would need table partitioning and read replicas. MongoDB natively supports sharding by shard key, making horizontal scaling straightforward. Neo4j Community Edition is single-instance; at production scale, workloads would migrate to Neo4j Enterprise or a distributed graph database such as Amazon Neptune.

### 6.3 Limitations

The dataset used is a snapshot; results may not generalise to streaming or real-time ingestion scenarios. All three databases were run in Docker containers on the same machine, which may introduce I/O contention during benchmarking and inflate execution times relative to dedicated deployments. The benchmark measures wall-clock time from a client process, which includes network round-trip to the Docker container, CLI tool start-up latency, and result serialisation overhead — none of which reflect pure query execution time inside the database engine.

## 7 Conclusion

This study implemented and benchmarked three data models for a real-world e-commerce dataset. Each database paradigm demonstrated strengths aligned with its design philosophy: PostgreSQL for structured aggregations with referential integrity, MongoDB for flexible document storage with embedded sub-documents, and Neo4j for graph-centric social network queries. The results confirm that the choice of data model should be driven by the dominant query patterns of the application rather than a single universal criterion.